上机试验 1：流水线性能分析 I

姓名：李祥熙　学号：2234412799

2026 年 4 月 24 日

1　实验目的

本实验围绕 5 级 MIPS 流水线展开，目标如下：

- 加深对 MIPS 五级流水线（IF/ID/EX/MEM/WB）工作机制的理解。
- 理解数据相关与控制相关对流水线停顿（stall）的影响。
- 通过设计不同指令组合，分别构造无冲突、RAW 冲突、分支跳转三种场景并进行性能分析。
- 对比定向路径（Forwarding）开启与关闭时的统计差异，理解硬件优化效果。

2　实验过程

2.1　实验环境

- 操作系统：Windows
- 汇编架构：MIPS（5 级流水）
- 模拟器 A：MIPSsim
- 模拟器 B：WinDLX（用于定向路径开/关对比）

2.2　实验步骤

1. 按照文档 `wdlx tutorial.pdf` 的说明，使用 win31 虚拟机，在启用和关闭 Forwarding 的两种情况下运行，并提供性能分析结果。
2. 编写并调试三份汇编程序：`no_hazard.s`、`raw_hazard.s`、`branch_jump.s`。
3. 在 MIPSsim 中分别加载运行，记录时钟周期图与统计信息。

4. 在 WinDLX 中进行 Forwarding enabled/disabled 两组运行并截取统计图。

5. 根据截图中的周期数、停顿类型与分支统计，完成图文分析。

3 实验内容

3.1 Win31 虚拟机：WinDXL 开启和关闭 Forwarding 的性能对比

Forwarding 功能通过在硬件中增加特殊的通路，允许指令在计算完成（通常是在执行阶段 EX 或访存阶段 MEM）后，直接将结果“转发”给流水线中后续需要的指令，而不必等待该结果写回到寄存器堆。

如果关闭 Forwarding 功能，为了保证逻辑正确，CPU 必须通过插入暂停周期 (Stalls) 来等待数据写回，产生大量的流水线空泡。而开启 Forwarding 功能则消除或减少了因“写后读（RAW)”相关引起的停顿，使指令流更顺畅。

按照文档 `wdlx tutorial.pdf` 的说明，在启用和关闭 Forwarding 功能的情况下运行示例程序 `fact.s` 计算 20 的阶乘，在程序运行结束后，查看运行的统计数据，比较两种情况下运行的周期数和停顿数区别。

3.2 场景一：无冲突流水线（no_hazard.s）

程序核心思路是先连续装载地址和数据，再进行两次加法与存储。关键点在于把可能的数据依赖拉开距离，尽量避免紧邻的 load-use 相关。

源码简要说明：

- 前 5 条 `ADDIU` 把数据标签地址装入 `$r8~$r12`。
- 三条 `LW` 将 `A,B,C` 读入寄存器。
- 两条 `ADD` 计算 `A+B` 和 `B+C`，再用 `SW` 写回结果。中间加入无关指令有意地拉开距离。
- 末尾 `TEQ $r0,$r0` 作为程序停止标记。

3.3 场景二：RAW 冲突（raw_hazard.s）

该程序故意构造了典型 load-use 依赖：先 `LW $r1,0($r9)`，紧接着 `ADD $r2,$r1,$r3` 读取 `$r1`，因此会触发 RAW 相关停顿。

源码简要说明：

- `INC` 与 `VAL` 分别加载到 `$r3` 和 `$r1`。
- `ADD` 直接消费刚 load 的 `$r1`，用于演示 RAW。
- `SW` 把结果写入 `OUT`，便于验证正确性。

3.4 场景三：分支跳转（branch_jump.s）

该程序包含 BEQ $r1,$r0,DO_ADD。由于 FLAG=0，分支被取走，形成一次明确的控制相关场景。

源码简要说明：

- 先读入 FLAG,X,Y。
- 当 FLAG==0 时跳转到 DO_ADD 执行 X+Y。
- 分支后仍执行 SW 与结束指令，便于在统计窗口观察控制停顿。

4 实验结果

4.1 Win31 虚拟机中的 WinDLX 结果：Forwarding 开关对比

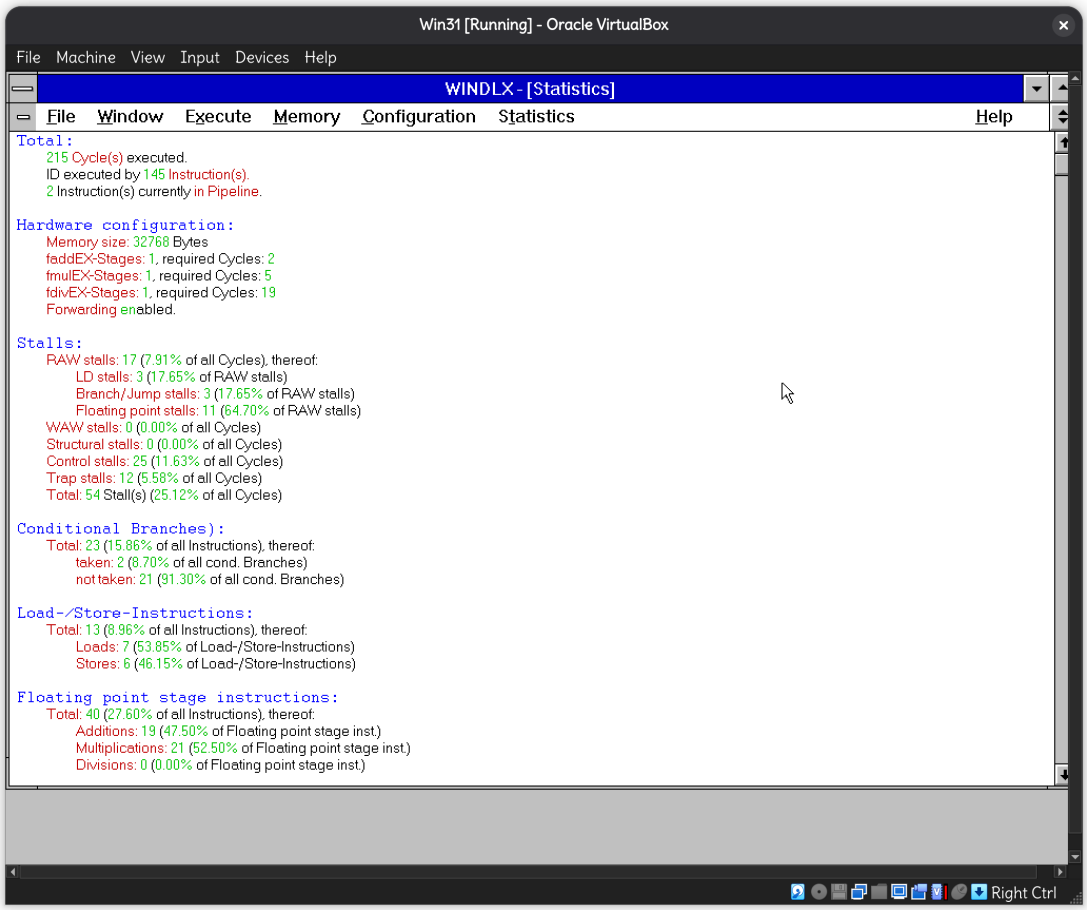

图 1: WinDLX 统计：Forwarding Enabled

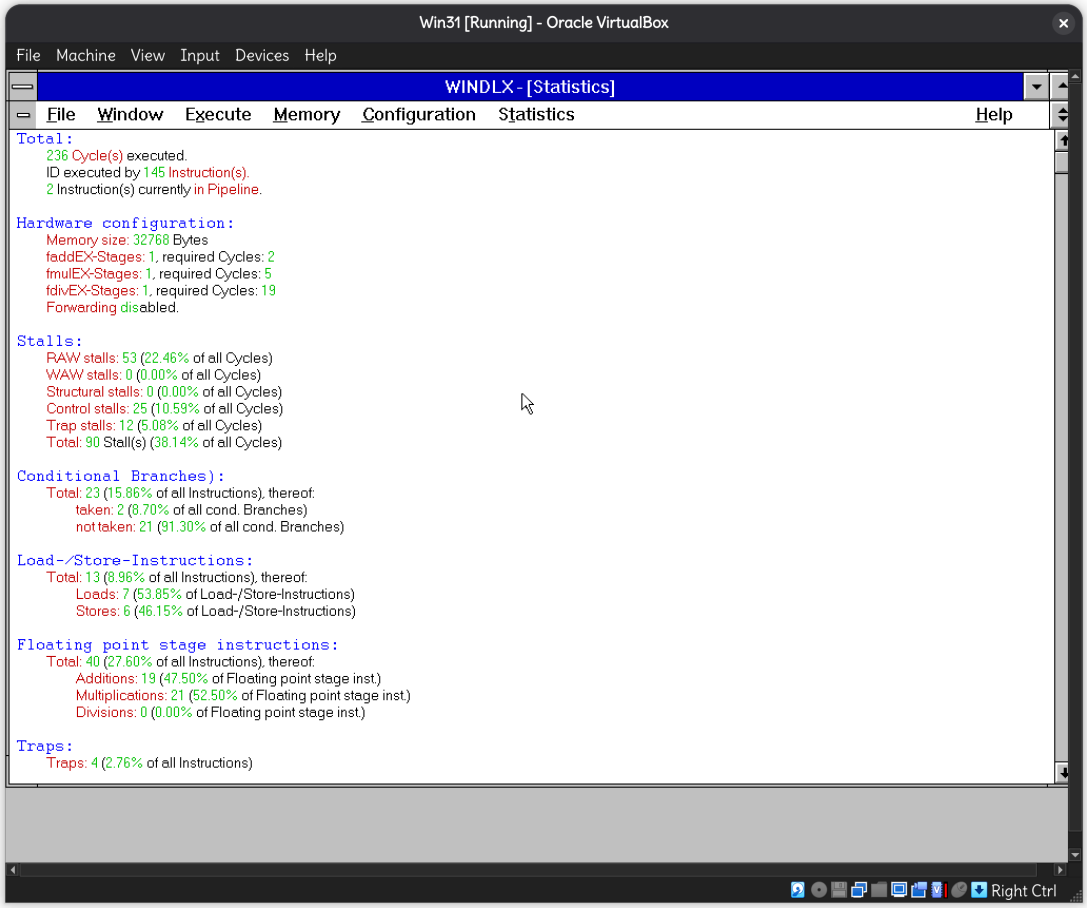

图 2: WinDLX 统计：Forwarding Disabled

对比分析：

- 开启定向路径：总周期 215，总停顿 54（25.12%），RAW 停顿 17（7.91%）。
- 关闭定向路径：总周期 236，总停顿 90（38.14%），RAW 停顿 53（22.46%）。
- 在同为 145 条已执行指令时，开启 Forwarding 减少 21 个周期，显著降低 RAW 相关停顿。

4.2 MIPSsim 结果：无冲突场景

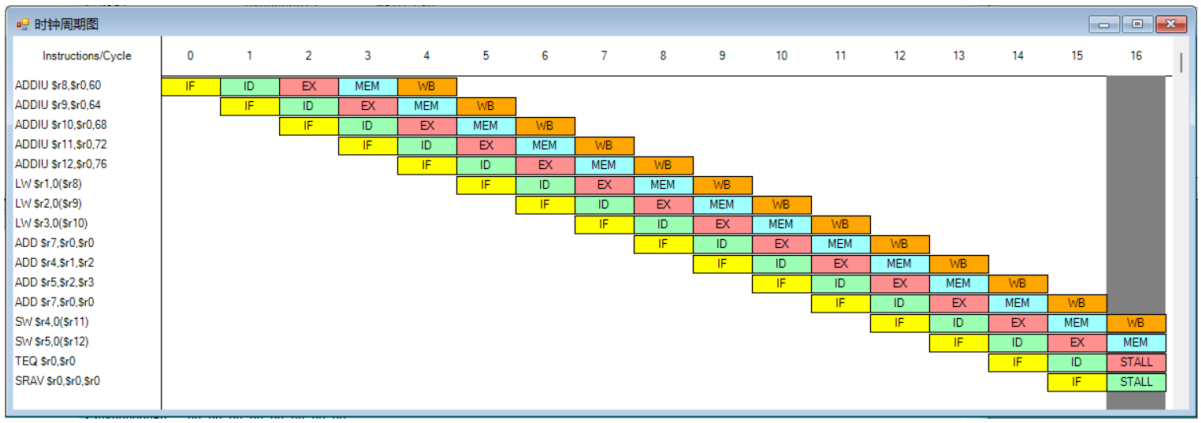

图 3: no_hazard.s 时钟周期图

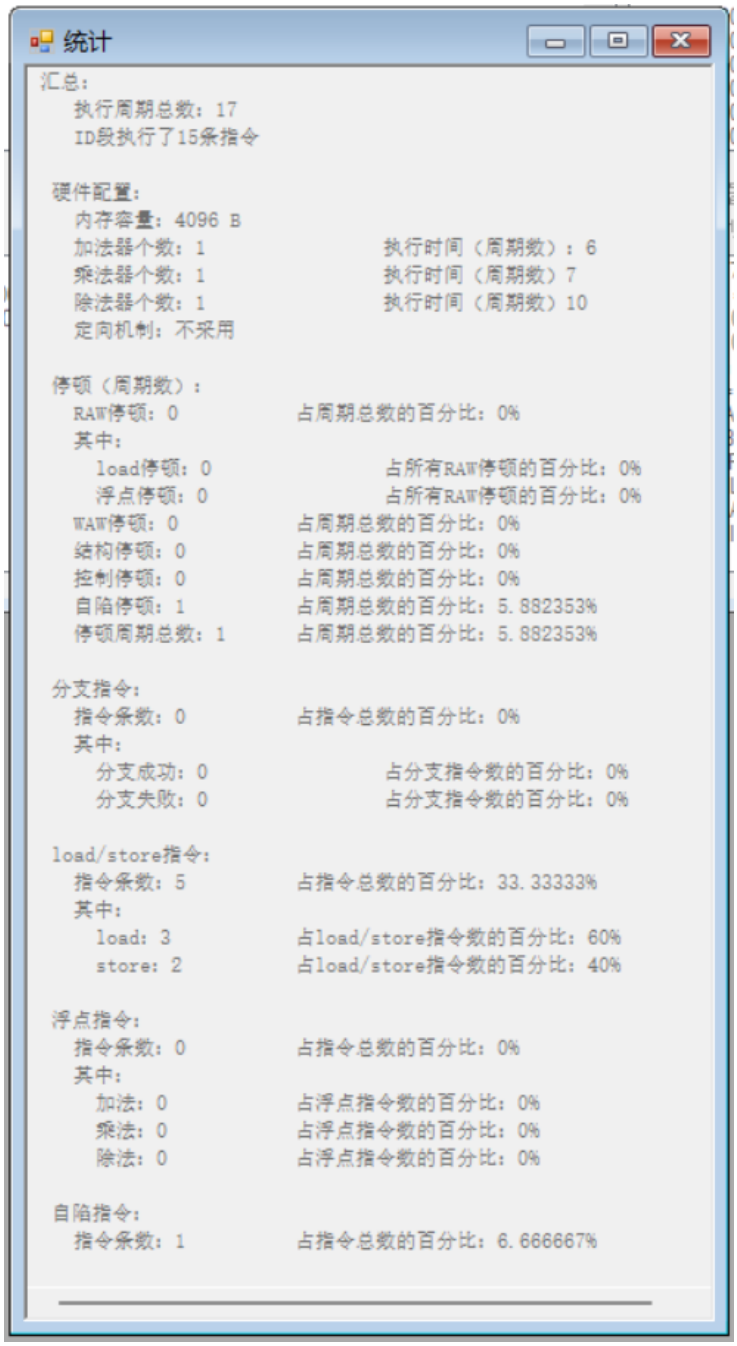

图 4: no_hazard.s 统计信息

观察与分析:

- 执行周期总数为 17，RAW 停顿为 0，控制停顿为 0。
- 仅出现程序结束相关停顿（停顿周期总数 1，占 5.882353%）。
- 满足“无冲突流水线演示”的实验要求。

4.3 MIPSsim 结果：RAW 冲突场景

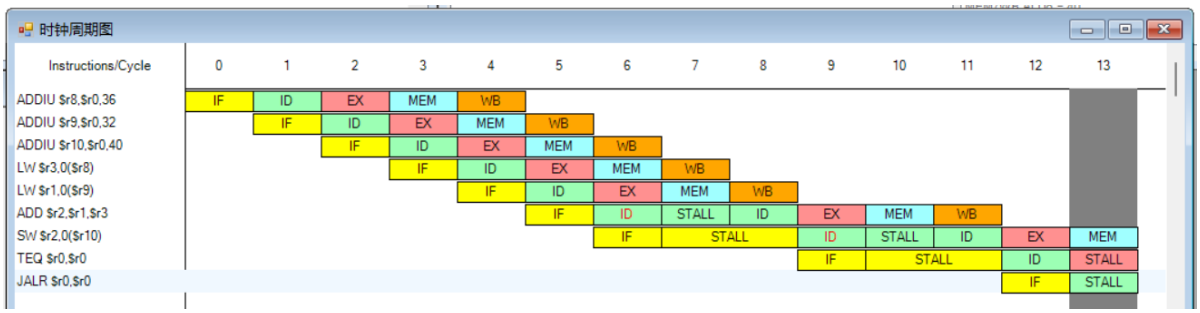

图 5: raw_hazard.s 时钟周期图

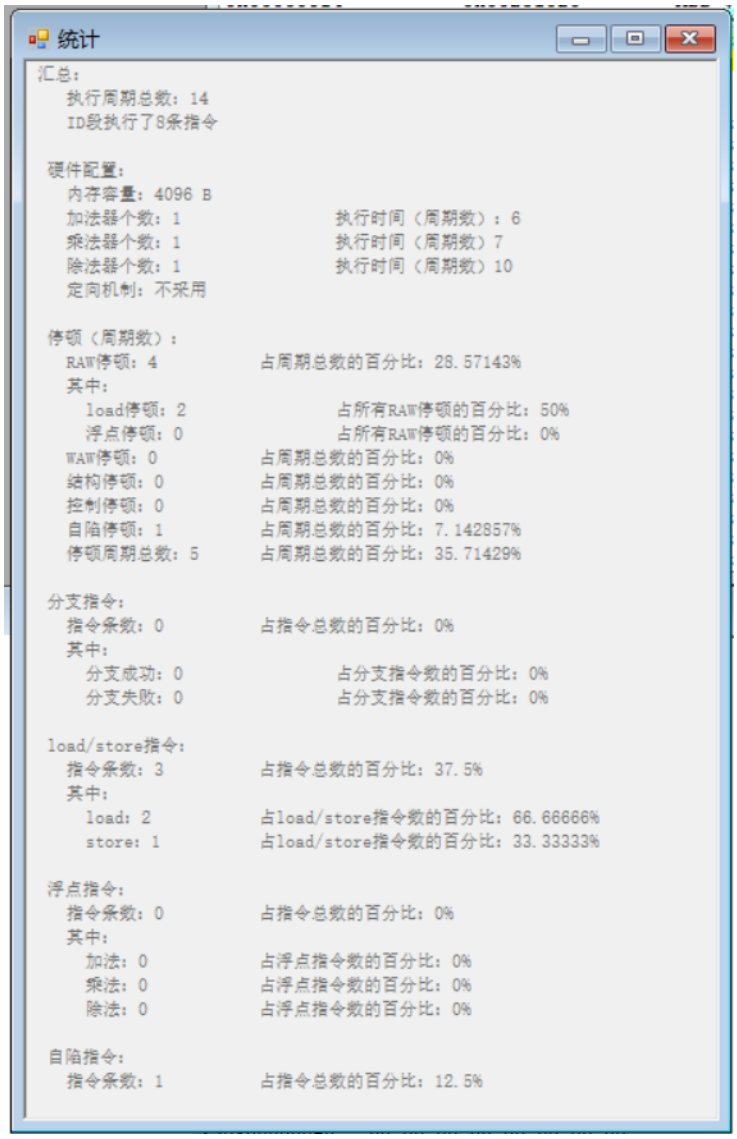

图 6: raw_hazard.s 统计信息

观察与分析：

- 执行周期总数为 14，出现 RAW 停顿 4 次（2 次为 load 停顿，占所有 RAW 停顿 50%）。
- 周期图中可见 `ADD` 和 `SW` 前后存在明显停顿，符合 load-use 冲突机理。

4.4 MIPSsim 结果：分支跳转场景

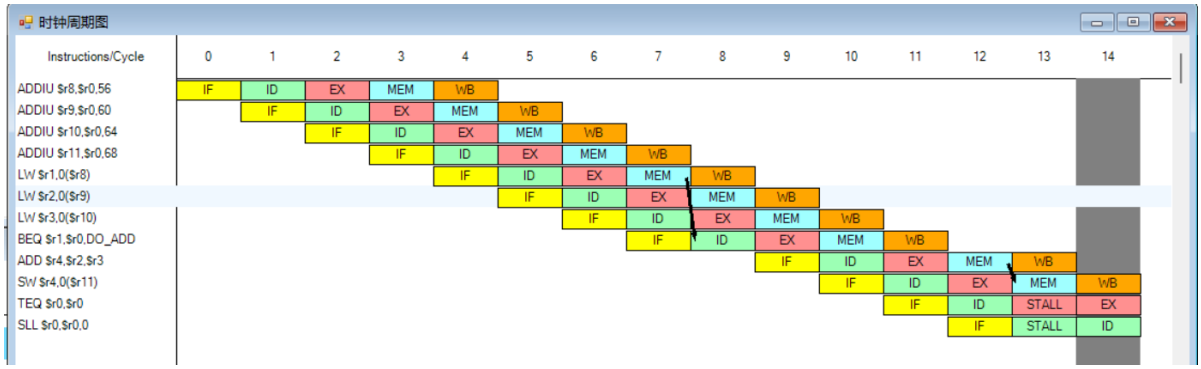

图 7: branch_jump.s 时钟周期图

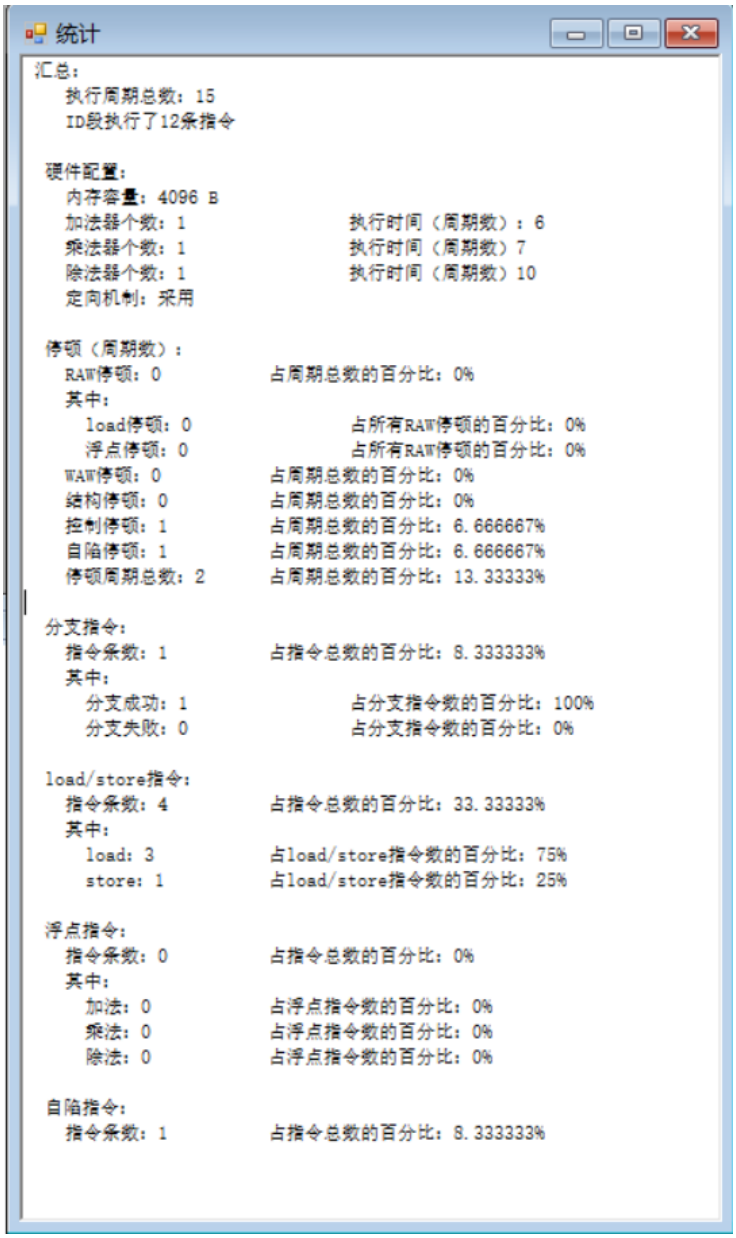

图 8: branch_jump.s 统计信息

观察与分析:

- 执行周期总数为 15，分支指令条数 1，分支成功 1 次（成功率 100%）。
- 控制停顿 1 次，占总周期 6.666667%。
- 总停顿周期 2（占 13.33333%），说明分支决策导致了流水线代价。

5 总结

本实验完成了实验指导书要求的性能分析和三类代码组合设计，并基于模拟器结果进行了分析:

- 通过 `no_hazard.s` 验证了可实现近似无数据冲突的流水执行。
- 通过 `raw_hazard.s` 验证了 load-use 型 RAW 冲突会引入停顿。
- 通过 `branch_jump.s` 验证了分支跳转会带来控制相关开销。
- 通过 WinDLX 对比验证了 Forwarding 在降低 RAW 停顿与总周期方面的有效性。

实验体会：在这次实验中，我掌握了 MIPS 模拟器的基本使用方法，更加直观地看到了程序运行过程中流水线的各个阶段。理解了流水线性能不仅取决于指令数量，还与指令编排、相关类型和硬件机制（如定向路径）紧密相关。合理调度可显著减少停顿，提高吞吐。

附录：实验源代码

A.1 no_hazard.s

```
.text
main:
ADDIU $r8,$r0,A
ADDIU $r9,$r0,B
ADDIU $r10,$r0,C
ADDIU $r11,$r0,RES1
ADDIU $r12,$r0,RES2
LW    $r1,0($r8)
LW    $r2,0($r9)
LW    $r3,0($r10)
ADD   $r7,$r0,$r0
ADD   $r4,$r1,$r2
ADD   $r5,$r2,$r3
SW    $r4,0($r11)
SW    $r5,0($r12)
TEQ   $r0,$r0

.data
A:    .word 7
B:    .word 11
C:    .word 5
RES1: .word 0
RES2: .word 0
```

A.2 raw_hazard.s

```
.text
main:
ADDIU $r8,$r0,INC
ADDIU $r9,$r0,VAL
ADDIU $r10,$r0,OUT
LW    $r3,0($r8)
LW    $r1,0($r9)
ADD   $r2,$r1,$r3
SW    $r2,0($r10)
TEQ   $r0,$r0

.data
VAL:  .word 9
INC:  .word 1
OUT:  .word 0
```

A.3 branch_jump.s

```
.text
main:
ADDIU $r8,$r0,FLAG
ADDIU $r9,$r0,X
ADDIU $r10,$r0,Y
ADDIU $r11,$r0,OUT
LW    $r1,0($r8)
LW    $r2,0($r9)
LW    $r3,0($r10)
BEQ   $r1,$r0,DO_ADD
ADD   $r4,$r0,$r0
SW    $r4,0($r11)
TEQ   $r0,$r0

DO_ADD:
ADD   $r4,$r2,$r3
SW    $r4,0($r11)
TEQ   $r0,$r0

.data
FLAG: .word 0
```

```
X:     .word 12
Y:     .word 30
OUT:   .word 0
```

A.4 运行说明

1) 在 MIPSsim 中分别加载并运行 no_hazard.s / raw_hazard.s / branch_jump.s。
2) 打开“时钟周期图”和“统计”窗口截图。
3) 在 WinDLX 中分别开启和关闭 Forwarding，记录 Statistics 页面。